

实验概览



⑧ 共包含八次实验，每个实验持续两周时间

- 每次实验会给出本次实验的具体要求，以及下次实验的简介和预备资料

⑧ 八次LAB分别为：

- Lab1 Syscall：熟悉risc-v和xv6中syscall的调用机制，尝试追踪xv6的系统调用过程并为xv6系统中添加新的系统调用
- Lab2 traps：熟悉risc-v和xv6的中断处理，并利用中断机制为xv6添加定时信号功能
- Lab3 thread：熟悉risc-v和xv6的进程切换实现，并为xv6添加用户态线程功能
- Lab4 lock：熟悉xv6中锁的实现，并对其进行改进以提高xv6的多核性能
- Lab5 pgtbl：熟悉risc-v和xv6中页表的实现，并利用页表实现新的功能
- Lab6 cow：熟悉xv6中的进程数据结构和虚拟内存处理，为xv6添加写时复制功能
- Lab7 fs：熟悉xv6中文件系统的实现，为xv6增加大文件和符号连接的功能
- Lab8 mmap：利用学期中学习到的虚拟内存以及文件系统的知识，为xv6实现mmap系统调用



实验概览



①环境准备：见canvas文件 [Lab参考资料/environment.pdf]

②部分参考资料（见canvas文件夹 [Lab参考资料]）

- Xv6 book: xv6的参考手册，包含xv6的具体实现以及背后原理
- risc-v手册：按需参考
 - riscv-privileged-20211203.pdf
 - riscv-spec-20191213.pdf
 - riscv-calling.pdf
- Git教程：
 - <http://rogerdudler.github.io/git-guide/index.zh.html>
 - <https://missing.csail.mit.edu/2020/version-control/>



Lab 1 Syscall



实验简介

- 熟悉risc-v和xv6中syscall的调用机制，尝试追踪xv6的系统调用过程并为xv6系统中添加新的系统调用

提前阅读

- Xv6 book Chapter1 and Chapter 2
- 阅读相关代码 `kernel/syscall.c` `kernel/syscall.h` `kernel/trap.c` `kernel/pipe.c` `user/usys.S`(编译后生成)

